《算法设计与分析》实验报告

实验四　回溯法

报告书			
姓名	顾凯琪	指导教师	方贤忠
学号	226001624	日　　期	2025/05/12
班级	22 级计算机 2 班		

实验内容

[要求先在 OJ 上提交通过]

1、[OJ]回溯法 1

2、[OJ]回溯法-堂练

实验目的

1）掌握回溯法的应用

2）学习掌握本类算法的设计与实现，在 OJ 上提交算法代码并测试通过，完成实验报告书写。

【如有查重雷同，请在此说明】

代码独立完成，无雷同。

【如有 OJ 未完成，请在此说明，并在本实验文档中补全】

自行完善：

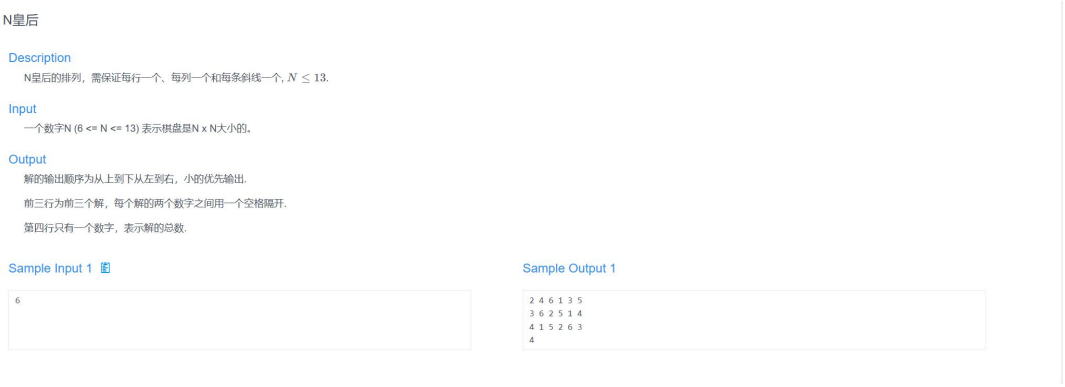
OJ 实验 1 作业正常时间完成。

OJ 实验 2 堂练全部完成。

实验过程和步骤

Module 1:　回溯法 1

【1、N 皇后 3（1-1000）】



N皇后

Description

N皇后的排列，需保证每行一个、每列一个和每条斜线一个, $N \leq 13$.

Input

一个数字N (6 <= N <= 13) 表示棋盘是N x N大小的。

Output

解的输出顺序为从上到下从左到右，小的优先输出.

前三行为前三个解，每个解的两个数字之间用一个空格隔开.

第四行只有一个数字，表示解的总数.

Sample Input 1

```
6
```

Sample Output 1

```
2 4 6 1 3 5
3 6 2 5 1 4
4 1 5 2 6 3
4
```

【2、程序代码及注释】

```
#include<bits/stdc++.h>
using namespace std;

vector<int> q(20, 0); // 存储每行皇后所在列号（q[i] 表示第 i 行的列号）
int solves = 0;       // 解的个数

// 回溯搜索第 i 行的皇后
void queen(int i, int n)
{
 if(i < n + 1){ // 还未放完所有皇后
  for(int col = 1; col <= n; col++){ // 尝试第 i 行的每一列
   bool flag = true;
   for(int j = 1; j < i; j++){ // 检查是否冲突
    if(col == q[j] || i + col == j + q[j] || i - col == j - q[j]){
     flag = false;
     break;
    }
   }

   if(flag){ // 若无冲突，放置皇后并继续搜索下一行
    q[i] = col;
    queen(i + 1, n);
```

```
    }
   }
  }
 else{ // 找到一个合法解
  solves++;
  if(solves <= 3){ // 输出前三个解
   for(int j = 1; j <= n; j++){
    cout << q[j] << (j == n ? "\n" : " ");
   }
  }
 }
}

void Solve(){
 int n;
 cin >> n;
 queen(1, n);          // 从第 1 行开始放置皇后
 cout << solves << endl; // 输出总解数
}

int main(){
 Solve();
 return 0;
}
```

【3、遇到的问题及解决方法】

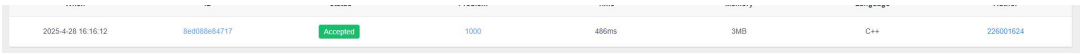
无。

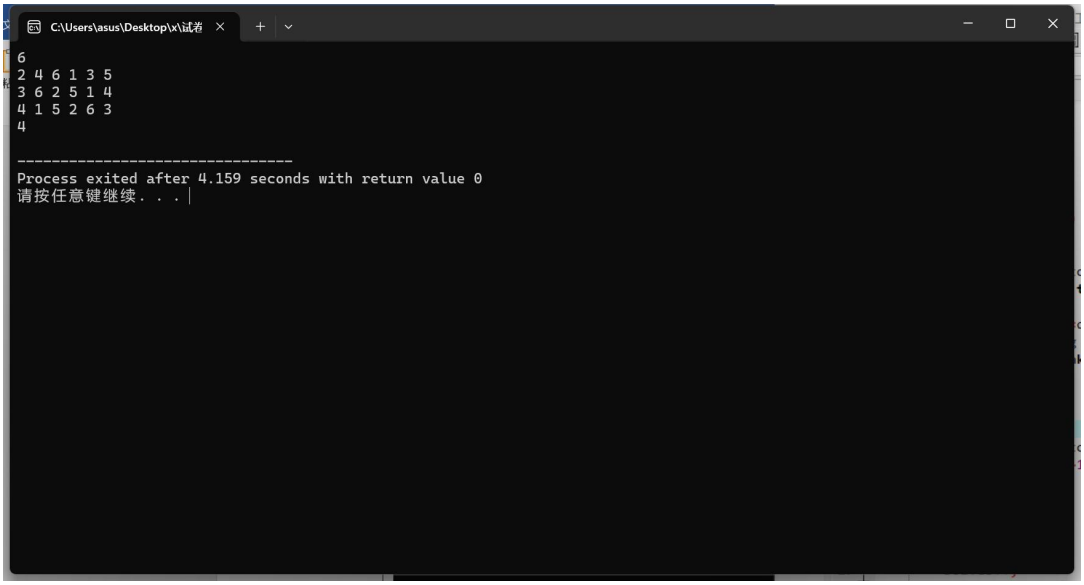
【4、算法分析与知识点】

本实验采用的回溯算法是一种通过递归搜索所有可能解的算法，它在解决问题的过程中会不断做出选择，如果发现已经做出的选择不能得到有效的解，则会撤销之前做出的选择，回溯到上一个选择点，继续尝试其他可能的选择。这种算法适用于解决组合问题，如 N 皇后问题。在 queen 函数中，我们首先要包含一个搜索过程，从第一行开始，逐行放置皇后。在每一行中，尝试将皇后放置在可能的列上。接着再进行冲突检测，对于当前行的每个可能的列位置，检查是否与之前放置的皇后冲突。冲突检测包括检查同一列、左上至右下的对角线、右上至左下的对角线。接着在进行递归放置，如果当前列没有冲突，则将皇后放置在该列，并递归地尝试放置下一个皇后。当所有皇后放置完毕(即 i>=n)，则找到了一个解，并增加解的个数，在解的个数小于等于三时，顺序输出每个方法中每一行皇后所在的列位置。如果在任何时候发现当前的选择不可能产生解（例如，当前行的所有列都已被前面的皇后占据），则进行剪枝操作，回溯并尝试其他选择。

时间复杂度为 O(n!)，因为最多需要尝试 n 个位置、每次递归进入下一层，并剪枝冲突情况；但由于剪枝有效，实际复杂度远小于 n!。空间复杂度为 O(n)，主要用于存储 q[1..n] 皇后的位置，递归栈深度最多为 n 层，故总空间也是线性级别。

【5、AC 截图及运行测试截图】





【1、符号三角形（1-1001）】

符号三角形

Description

符号三角形的 第1行有n个由"+"和"-"组成的符号，以后每行符号比上行少1个，2个同号下面是"+"，2个异号下面是"-"。计算有多少个不同的符号三角形，使其所含"+"和"-"的个数相同。

Input

整数n(n<=20)

Output

符合要求的符号三角形的个数

Sample Input 1

15

Sample Output 1

1896

【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

int train[21][21]; // 模拟下三角形区域
int half = 0;      // 1 的数量目标值（等于总格子数的一半）
int count1 = 0;    // 当前已经填入的 1 的数量
int sum = 0;       // 合法方案计数
int n = 0;         // 输入的 n 值

// 回溯函数，start 表示当前填充的列（从上到下）
void traceback(int start, int end) {
    if (start == end) {
        if (count1 == half) sum++; // 如果 1 的数量恰好为 half，则
```

```
    是一个合法解
            return;
        }

        // 枚举当前列顶端放 0 或 1
        for (int i = 0; i < 2; i++) {
            train[0][start] = i;
            count1 += i;

            // 根据异或规则推导出当前列剩余的值
            for (int j = 1; j <= start; j++) {
                train[j][start - j] = !(train[j - 1][start - j] ^ t
    rain[j - 1][start - j + 1]);
                count1 += train[j][start - j];
            }

            // 剪枝：当前已放 1 的数量不超过目标，且剩余格子足以放满剩余 1
            if (count1 <= half && ((start + 1) * start / 2 - count1
    ) <= half) {
                traceback(start + 1, end);
            }

            // 回溯还原状态
            for (int j = 1; j <= start; j++) {
                count1 -= train[j][start - j];
            }
            count1 -= i;
        }
    }

    int main() {
        cin >> n;

        // 特殊情况处理
        if (n == 0) {
            cout << 0 << endl;
            return 0;
        }

        // 总格子数为 (n+1)*n/2，若为奇数无法平均分配 0/1，输出 0
        if ((n * (n + 1)) / 2 % 2) {
            cout << 0 << endl;
        } else {
```

```
        half = (n + 1) * n / 4; // 目标 1 的数量
        traceback(0, n);
        cout << sum << endl;
    }

    return 0;
}
```

【3、遇到的问题及解决方法】

if (start == end) {

if (count1 == half) sum++;

return;

}

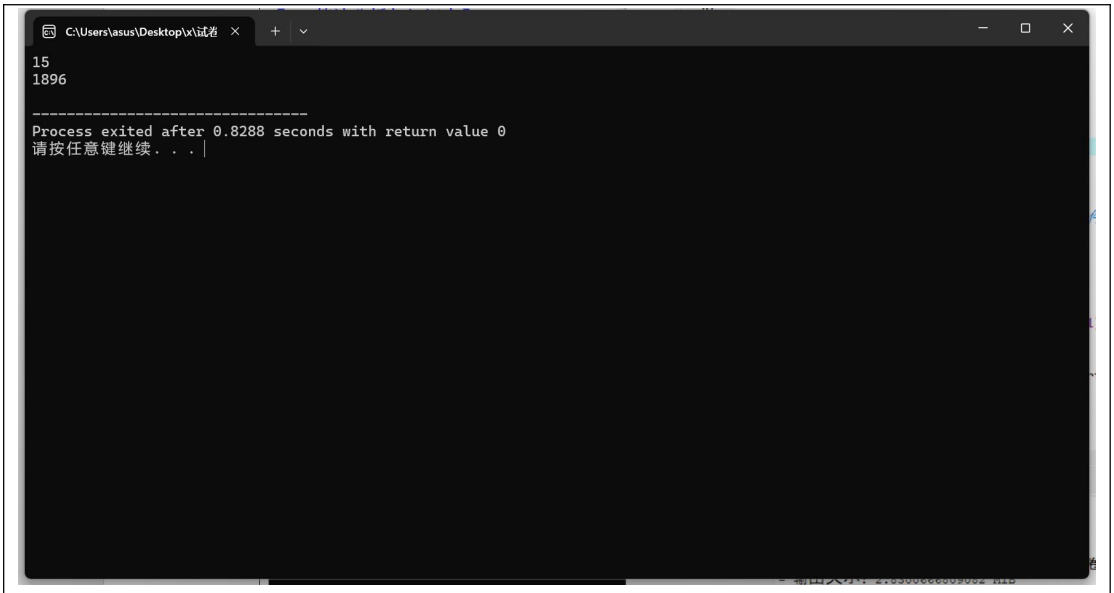
这里原先把两个判断条件放一起了，用 and 符号进行连接，导致函数无法正确返回，因为 count1 其实并未到达 half，后把条件分开判断即可。

【4、算法分析与知识点】

在本次实验中，我们首先对题目进行分析，可以得到，只要我们确定下第一行中符号的类别，便可以得到整个三角形的样貌。而第一行每多一个符号相当于在斜向增加了一行符号。回溯算法是一种通过递归调用来探索所有可能解的算法。在该题中，我们采用递归函数 Dfs 尝试构建符号三角形的每一行，直到满足解或者找到解。其次，我们可以通过减枝来减少不必要的计算，提高算法效率，在该题中，我们通过 if(count <= half && (t+2)*(t+1)/2 - count <= half)进行判断，若当前任意符号的数量超过了总量的一半，则说明不符合条件，则不需要继续探索该路径。count 变量用于跟踪当前三角形中 "-" 的数量。当构建完所有行并且 "+" 和 "-" 的数量相等时，sum 增加，表示找到一个有效的解。

该程序通过回溯法构造满足异或规则和 1 的数量为总格子数一半的三角形填充方案。时间复杂度在最坏情况下为 $O(2^n)$，因为每列顶端有两个选择（0 或 1），共有 n 列；但由于异或规则决定了其余位置，且剪枝策略有效，实际搜索树远小于 2^n。空间复杂度为 $O(n^2)$，主要用于 train[n][n] 数组存储三角形区域的状态。

【5、AC 截图及运行测试截图】



【1、马的遍历（1-1002）】



马的遍历

Description

中国象棋半张棋盘有九条纵线(0,1,2,3,...,8)，5条横线（0,1,2,3,4）。马自左下角往右上角跳。规定只许往右跳，不许往左跳。

如下所示为一条跳行路线：

0,0->2,1->3,3->1,4->3,5->2,7->4,8

Input

无

Output

打印所有可能的路径（按0、1、2、3、4...顺序深度遍历），每行一条路径，按顺序输出马跳过的各点坐标。

Sample Input 1

无

Sample Output 1

```
0,0->2,1->4,2->3,4->1,5->3,6->4,8
0,0->2,1->4,2->3,4->1,5->2,7->4,8
0,0->2,1->4,2->2,3->4,4->3,6->4,8
0,0->2,1->4,2->2,3->4,4->2,5->4,6->2,7->4,8
...
...
```

【2、程序代码及注释】

```
#include <bits/stdc++.h>
using namespace std;

int n = 4, m = 8;          // 目标坐标 (4,8)
int a[100][10];            // 存储路径的坐标信息
int i = 1;                 // 当前路径位置索引

// 马步移动的 DFS 路径搜索
void dfs(int y, int x)
{
    if (y < 0 || y > n || x > m) {   // 越界剪枝
        return;
    }

    if (y == n && x == m) {          // 成功到达终点
        for (int j = 1; j < i; j++) {
```

```
            cout << a[j][1] << "," << a[j][2] << "->";
        }
        cout << "4,8" << endl;        // 打印最终坐标
    }
    else {
        a[i][1] = y;                 // 记录当前位置
        a[i][2] = x;
        i++;

        // 模拟马的四种跳法（只考虑向右和上方，避免死循环）
        dfs(y + 2, x + 1);
        dfs(y + 1, x + 2);
        dfs(y - 1, x + 2);
        dfs(y - 2, x + 1);

        // 回溯恢复状态
        a[i][1] = 0;
        a[i][2] = 0;
        i--;
    }
}

int main()
{
    dfs(0, 0);   // 从原点 (0,0) 开始搜索
    return 0;
}
```

【3、遇到的问题及解决方法】

Wrong Answer: 在进行回溯算法时，当深度优先搜索算法搜到最后结果时再将路径上搜索的点集输出，而不应该在搜索过程中进行输出，因为这样会将一些回溯过程的路径输出，导致答案错误。

【4、算法分析与知识点】

在本题中已经限定马的前进方向，并且已经直到终点与起点，所以我们采用深度优先算法进行解决。深度优先搜索是一种用于遍历或搜索树或图的算法。它从一个节点开始，尽可能深地搜索树地分支。当路径的末端节点无法进一步拓展时，算法会回溯到最近的分治节点，继续搜索。

递归函数 dfs(int y,int x),这个函数接受当前马的坐标(x,y)作为参数，并递归地禅师所有可能的走法。

边界条件：如果当前坐标超出棋盘边界(y<0||y>n||x>m)，则停止搜索。

目标条件：如果当前坐标等于目标坐标(y == n && x==n)，则打印出从起点到当前点的路径。

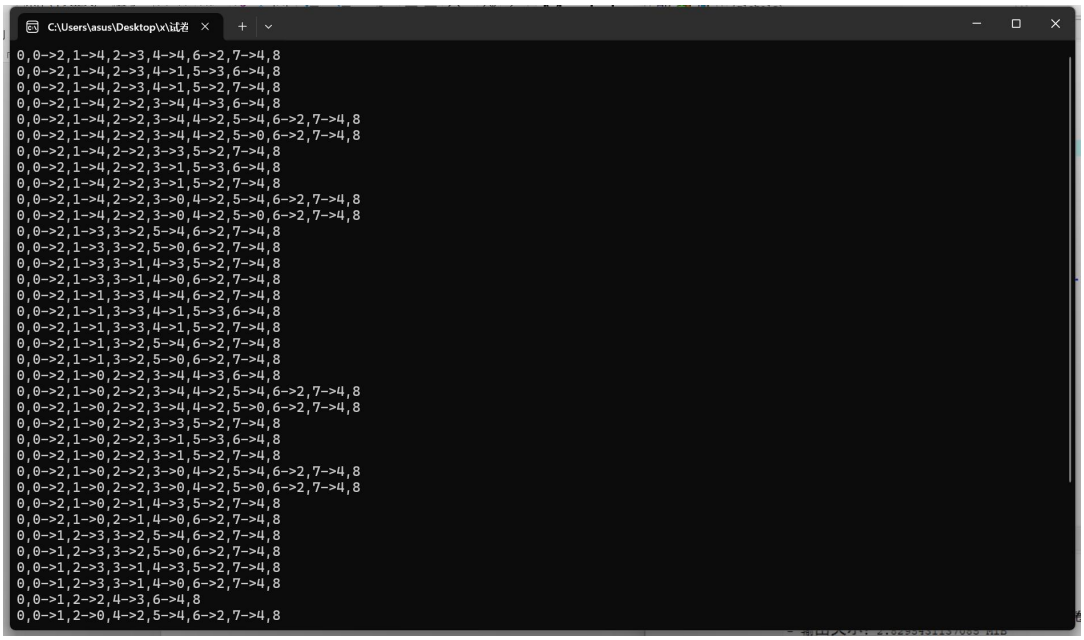
走法尝试：一共有四种走法，在符合条件的情况下均要进行尝试

回溯：在尝试了所有可能的走法后，算法需要回溯以尝试其他路径。这通过重置当前节点的值并减少步数 i 来实现。

该程序使用深度优先搜索（DFS）枚举从起点 (0,0) 到终点 (4,8) 的所有“马步”路径。每一步最多有 4 种跳法，因此时间复杂度在最坏情况下为 O(4^k)，其中 k 是步数上限（近似为终点坐标之和，这里最多约 12 步），但由于剪枝限制在棋盘内，实际搜索空间远小于理论值。空间复杂度为 O(k)，用于递归栈和路径记录数组 a 的存储。

【5、AC 截图及运行测试截图】





Module 2:　回溯法-堂练

【1、地图着色（2-1000）】

地图着色

Description

给定图G=(V, E)，需要为图G的各顶点着色，是否有一种着色法使G中相邻的两个顶点有不同的颜色.

Input

第一行是顶点的个数n（2≤n≤8），颜色数m（1≤m≤n）。

接下来是顶点之间的连接关系: a b; 表示a和b相邻。顶点从1开始计。

当a, b同时为0时表示输入结束。

Output

输出着色方案总数,以及最少颜色数。如果无可行方案，颜色数为0。

Sample Input 1

```
5 4
1 3
1 2
1 4
2 3
2 4
2 5
3 4
4 5
0 0
```

Sample Output 1

```
48 4
```

【2、程序代码及注释】

```
#include<bits/stdc++.h>
using namespace std;

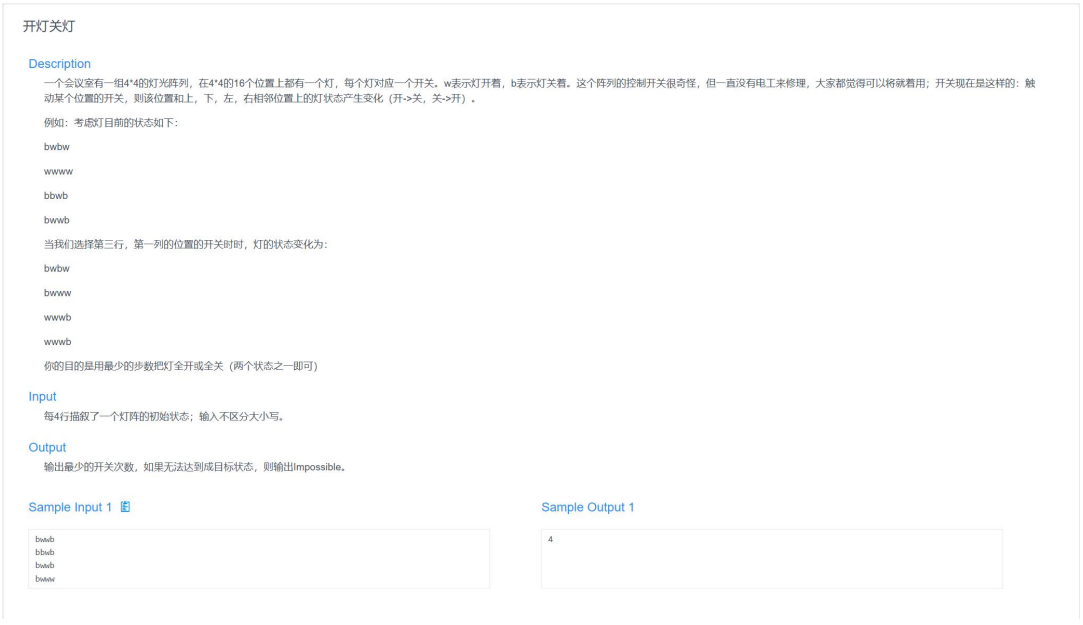
int n,m;
int place[10][10] = {0};
vector<int> coled(10,0);
int mincl = 11;
int count1 = 0;

void traceback(int i,int maxcl){
 if(i <= n){
  if(maxcl > m) return;
  for(int col = 1;col<=n;col++){
   bool flag = true;
   for(int j = 1;j<i;j++){
    if(coled[j] == col && place[i][j] == 1){
     flag = false;
     break;
    }
   }
   if(flag){
    coled[i] = col;
    if(col>maxcl){
     maxcl++;
    }
    traceback(i+1,maxcl);
    coled[i] = 0;
   }
  }
 }
 else{
  if(maxcl > m) return;
  if(maxcl<mincl){
   mincl = maxcl;
  }
  count1++;
  return;
 }
}


```

```
int main(){
 cin>>n>>m;
 while(1){
  int a,b;
  cin>>a>>b;
  if(a ==0 && b == 0){
   break;
  }
  else{
   place[a][b] = 1;
   place[b][a] = 1;
  }
 }
 traceback(1,1);
 if(count1 == 0) cout<<count1<<" "<<0<<endl;
 else cout<<count1<<" "<<mincl<<endl;
}
```

【1、开灯关灯（2-1001）】



开灯关灯

Description

一个会议室有一组4*4的灯光阵列，在4*4的16个位置上都有一个灯，每个灯对应一个开关。w表示灯开着，b表示灯关着。这个阵列的控制开关很奇怪，但一直没有电工来修理，大家都觉得可以将就着用；开关现在是这样的：触动某个位置的开关，则该位置和上，下，左，右相邻位置上的灯状态产生变化（开->关，关->开）。

例如：考虑灯目前的状态如下：

bwbw

wwww

bbwb

bwwb

当我们选择第三行，第一列的位置的开关时时，灯的状态变化为：

bwbw

bwww

wwwb

wwwb

你的目的是用最少的步数把灯全开或全关（两个状态之一即可）

Input

每4行描叙了一个灯阵的初始状态；输入不区分大小写。

Output

输出最少的开关次数，如果无法达到成目标状态，则输出Impossible。

Sample Input 1

```
bwwb
bbwb
bwwb
bwww
```

Sample Output 1

```
4
```

【2、程序代码及注释】

```
#include<bits/stdc++.h>
using namespace std;

char st[5][5], tmp[5][5];
int dx[5] = {0, 0, 1, -1, 0};
int dy[5] = {1, -1, 0, 0, 0};
```

```cpp
int ans = 1e9;

void flip(int x, int y){
    for(int i = 0; i < 5; i++){
        int nx = x + dx[i];
        int ny = y + dy[i];
        if(nx >= 1 && nx <= 4 && ny >= 1 && ny <= 4){
            tmp[nx][ny] = tmp[nx][ny] == 'b' ? 'w' : 'b';
        }
    }
}

bool check(){
    char c = tmp[1][1];
    for(int i = 1; i <= 4; i++)
        for(int j = 1; j <= 4; j++)
            if(tmp[i][j] != c) return false;
    return true;
}

void dfs(int k, int cnt){
    if(k == 16){
        if(check()) ans = min(ans, cnt);
        return;
    }
    int x = k / 4 + 1;
    int y = k % 4 + 1;

    // 不按当前灯
    dfs(k + 1, cnt);

    // 按当前灯
    flip(x, y);
    dfs(k + 1, cnt + 1);
    flip(x, y);
}

int main(){
    for(int i = 1; i <= 4; i++)
        for(int j = 1; j <= 4; j++){
            cin >> st[i][j];
            st[i][j] = tolower(st[i][j]);
        }
```

```
50.
51.     memcpy(tmp, st, sizeof(st));
52.     dfs(0, 0);
53.
54.     if(ans == 1e9) cout << "Impossible" << endl;
55.     else cout << ans << endl;
56.
57.     return 0;
58. }
```

【1、子集和问题（2-1002）】

子集和问题

Description

子集和问题的一个实例为(S,c)。其中，$S=\{x_1,x_2,\dots,x_n\}$是一个正整数的集合，

c是一个正整数。子集和问题判定是否存在S的一个子集S_1，使得

$\sum_{x\in S_1}x=c$. 试设计一个解子集和问题的回溯法。

Input

第 1 行有 2 个正整数 n 和 c，n 表示 S 的大小，c是子集和的目标值。接下来的 1 行中，有 n 个正整数，表示集合 S 中的元素。

Output

当有解时，输出 YES!；当问题无解时，输出No solution! .

Sample Input 1

```
5 10
2 2 6 5 4
```

Sample Output 1

```
YES!
```

【2、程序代码及注释】

```
1. #include<bits/stdc++.h>
2. using namespace std;
3.
4. int n, c;
5. vector<int> nums;
6. bool flag = false;
7. unordered_set<string> vis;
8.
9. void traceback(int i, int sum1){
10.  if(flag || sum1 > c) return;
11.  if(sum1 == c){
12.   cout << "YES!" << endl;
13.   flag = true;
14.   return;
15.  }
16.  if(i >= n) return;
17.
18.  string state = to_string(i) + "-" + to_string(sum1);
19.  if(vis.count(state)) return;
20.  vis.insert(state);
```

```
21.
22.  traceback(i + 1, sum1 + nums[i]);
23.  traceback(i + 1, sum1);
24.}
25.
26.int main(){
27.  cin >> n >> c;
28.  int sum = 0, x= INT_MAX;
29.  for(int i = 0; i < n; i++){
30.   int a;
31.   cin >> a;
32.   nums.push_back(a);
33.   sum+=a;
34.   x = min(a,x);
35.  }
36.  if(sum < c ||x > c){
37.   cout << "No solution!" << endl;
38.   return 0;
39.  }
40.  traceback(0, 0);
41.  if(!flag) cout << "No solution!" << endl;
42.}
```

【第二部分】

【同类题型实验扩展/可选/会影响分值】

1、题目+测试数据+结果

2、分析

3、建议：难度不低于实验题

【1、tfs-分考场】

n 个人参加某项特殊考试。

为了公平，要求任何两个认识的人不能分在同一个考场。

求至少需要分几个考场才能满足条件。

【2、程序代码及注释】

```
1. #include<bits/stdc++.h>
2. using namespace std;
3.
4. int M[101][101]={0}; //关系矩阵
5. int room[101][101]={0}; //考场矩阵
6. int ans=99999; //存放最优解
7. int n,m;
```

```
8.
9. void dfs(int num,int kc);
10.
11.
12.int main()
13.{
14. cin>>n>>m; //n 代表参加考试的人数,m 代表接下来输入 m 行数据来代表两两间关系
15. int a,b;
16. for(int i=0;i<m;i++){
17.  cin>>a>>b;
18.  M[a][b]=1;
19.  M[b][a]=1;
20. }
21. dfs(1,1);
22. cout<<ans<<endl;
23. return 0;
24.}
25.
26.void dfs(int num,int kc) //num 表示当前正在考虑的学生编号，kc 表示当前的考场数
27.{
28. if(kc>=ans)return ; //剪枝
29. if(num>n){ //更新
30.  ans=kc;
31. }
32. else{
33.  for(int i=1;i<=kc;i++){
34.   int k=0;
35.   while(room[i][k]&&M[num][room[i][k]]==0){ //该位置有人且该人与当前学生没有关系则继续找空位
36.    k++;
37.   }
38.   if(room[i][k]==0){ //在该考场中没有关系且找到空位
39.    room[i][k]=num;//入考场
40.    dfs(num+1,kc);//递归
41.    room[i][k]=0;//回溯
42.   }//如果有关系就不能在一个考场，跳出循环找下一个考场
43.  }
44.  room[kc+1][0]=num;//所有考场中都存在具有关系的人，则需要新开设一个考场
45.  dfs(num+1,kc+1);//递归
46.  room[kc+1][0]=0;//回溯
```

```
47. }
48.}
```

【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

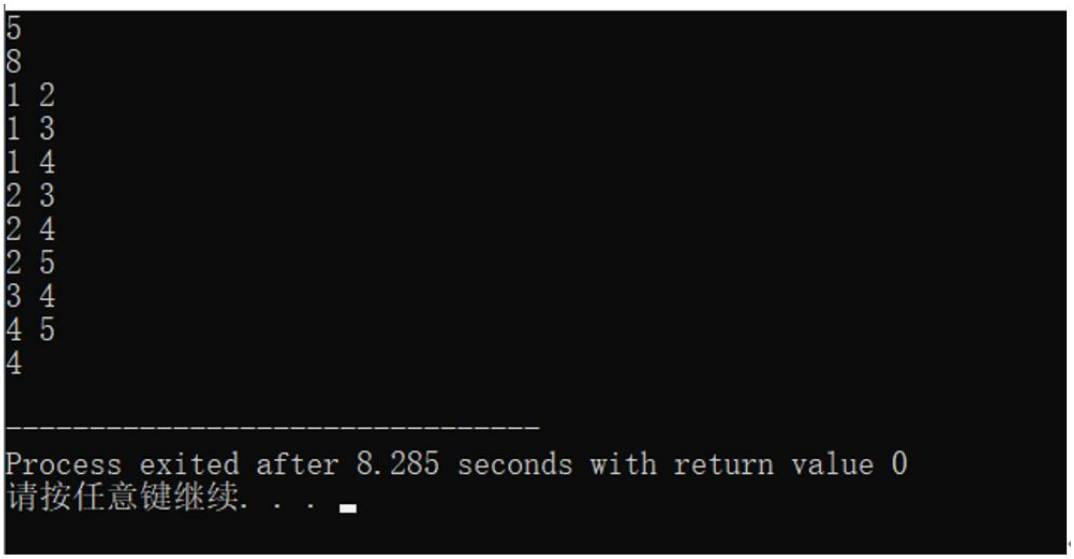
首先对本题目进行分析，本次实验的目标是分配最小的考场数，而分配的限定条件为某一考场内不存在两两事先相互认识的考生。故我们采用 dfs 算法来分配考场以满足条件并遍历所有学生。

在 dfs 中首先进行剪枝判断，若当前已分配的考场数不小于已计算出的最少考场数，则进行剪枝。其次，如果 num>n，则代表所有的学生均已经分配到考场，故可以更新 num 的值。

在考生座位还未完全分配完时，我们使用两层循环来进行遍历，第一层循环来遍历各个考场，就是子集树，来查看将考生分配至各个考场的结果。第二层循环就是用于判断考生是否能进入该考场。若能进入考场，则将考生数+1 进行深搜 dfs(num+1,kc)。深搜结束后进行回溯 room[i][k]=0。

若是一层循环遍历后仍然未找到可加入的教室，则将考生数与教室数均+1 处理后进行深搜 dfs(num+1,kc+1)。深搜结束后进行回溯 room[kc+1][0]=0。

【5、AC 截图及运行测试截图】



【1、 森林迷宫】

迷宫可以看成是由 n * n 的格点组成，每个格点只有 2 种状态：^ 和 # ；前者表示可以通行后者表示不能通行。

她只能移动到东南西北(或者说上下左右)四个方向之一的相邻格点上，

【2、程序代码及注释】

```
1. #include <bits/stdc++.h>
```

```cpp
using namespace std;

int graph[101][101];
int n = 0;

//把字符图转成数字图储存起来
void get_graph(){
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            char a;
            cin >> a;
            if(a == '^') graph[i][j] = 1;
            else if(a == '#') graph[i][j] = 0;
        }
    }
}

//爆搜一下，你就知道
bool dfs(int x1, int y1, int x2, int y2){
 int a[4] = {-1, 0, 1, 0};
 int b[4] = {0, -1, 0, 1};
    graph[x1][y1] = -1;  //-1 代表这个格子位于已经走过的路途上
    if(x1 == x2 && y1 == y2) return true;
    for(int i = 0; i < 4; i++){  //i 分别表示向左，向下，向右，向上移动
        if(x1 + a[i] >= 0 && x1 + a[i] < n && y1 + b[i] >= 0 && y1 + b[i] < n && graph[x1 + a[i]][y1 + b[i]] == 1) //若移动方向非越界并且并且可通行
            if(dfs(x1 + a[i], y1 + b[i], x2, y2)) return true;
    }
    graph[x1][y1] = 1; //回溯，将这一部分重置为可通行
    return false;
}


int main(){
    int k = 0;
    cin >> k;
    //读入 K 组数据并给出相应结果
    for(int i = 0; i < k; i++){
        cin >> n;
        get_graph();
        int x1, y1, x2, y2;
```

```
42.      cin >> x1 >> y1 >> x2 >> y2;
43.      if(!graph[x1][y1] || !graph[x2][y2]){ //起点或者终点有一个不可通行则返回false
44.       cout<<"NO"<<endl;
45.   }else if(dfs(x1, y1, x2, y2)) {  //深搜，若成功返回yes
46.       cout<<"YES"<<endl;
47.   }else{
48.       cout<<"NO"<<endl;
49.   }
50.     }
51.     return 0;
52. }
```

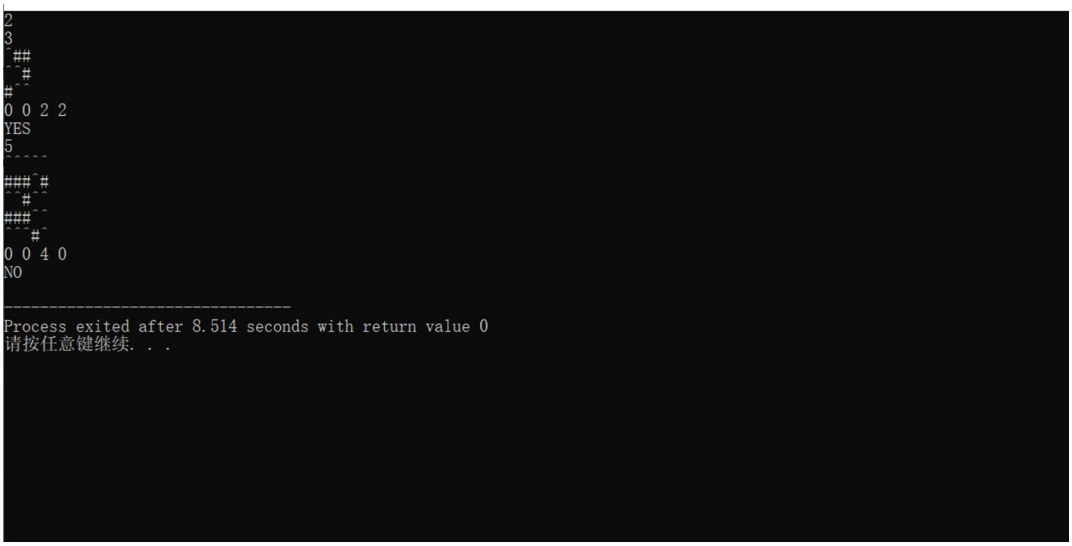
【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

在本次实验中，首先要实现数据的输入和预处理。首先，我们读取测试用例的数量 k。对于每个测试用例，进行图的录入通过 get_graph 函数将字符表示的迷宫转换为数字表示，其中 1 表示可通行的格点，0 表示不可通行的格点。接着再读取起点与终点的坐标。函数使用四个方向的移动向量 a 和 b 来尝试向四个基本方向（上、下、左、右）移动。如果当前格点变为不可通行或者越界，则不会继续探索该路径。如果到达终点，则返回 true，表示找到了一条路径。如果在当前格点没有找到路径，则回溯，将当前格点的状态恢复为可通行。在开始搜索之前，程序检查起点和终点是否可通行，如果不可行，则直接输出 "NO"。如果 dfs 函数成功找到路径，则输出 "YES"；否则，输出 "NO"。

【5、AC 截图及运行测试截图】



【1、tfs-花花草草】

数值 1 表示该区域种着花，数值 0 表示该区域是草地。

如果花的上下左右也是花，则这些连着的花必须是同一种花，不相连的花的品种不同。

【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

int M;
int lawn[100][100];
int visited[100][100]={0};

void dfs(int row, int col) {  // 深度搜索
    visited[row][col] = 1;   //代表这个节点被搜索过
    if (row > 0 && lawn[row - 1][col] == 1 && visited[row - 1][col] == 0)  //往左侧节点搜索
  dfs(row - 1, col);
    if (row < M - 1 && lawn[row + 1][col] == 1 && visited[row + 1][col] == 0)  //往右侧节点搜索
  dfs(row + 1, col);
    if (col > 0 && lawn[row][col - 1] == 1 && visited[row][col - 1] == 0)   //往上方节点搜索
  dfs(row, col - 1);
    if (col < M - 1 && lawn[row][col + 1] == 1 && visited[row][col + 1] == 0)  //往下方节点搜索
  dfs(row, col + 1);
}

int main() {
    cin >> M;
    for (int i = 0; i < M; i++) {  //建图
        for (int j = 0; j < M; j++) {
            cin >> lawn[i][j];
        }
    }
    int count = 0;
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < M; j++) {
            if (lawn[i][j] == 1 && visited[i][j] == 0) { //如果是花且未被搜索过，则花的品种+1
                count++;
                dfs(i, j);  //深搜，排除同一种类的花的数量
            }
        }
    }
    cout << count << endl;
    return 0;
}
```

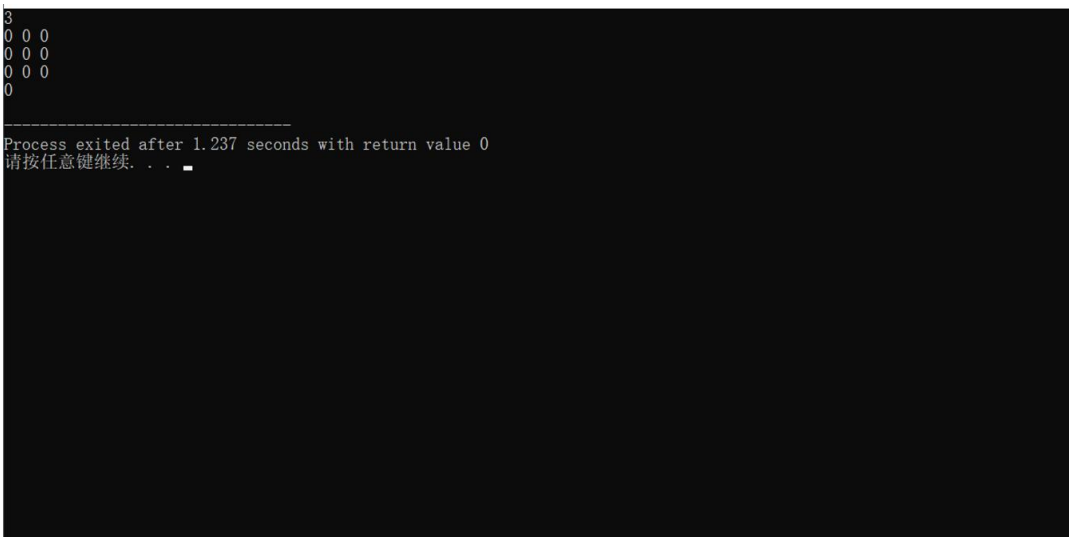
【3、遇到的问题及解决方法】

Wrong Answer:原先并未采用回溯算法，而仅仅只是简单的进行了 for 循环，算法存在问题。

【4、算法分析与知识点】

针对该问题，我们首先要完成图的构建，使用二维数组来构建图，其中 lawn[a][b] 为 1 表示该节点为花，为 0 表示该节点为草地。接着我们使用深度优先搜索(DFS)算法从当前位置(row,col)开始，递归地标记所有连通的花。在 dfs 函数中，首先将当前位置标记为已访问（visited[row][col] = 1），以避免重复计数。程序检查当前位置的上下左右四个相邻区域，如果相邻区域是花且未被访问过，则递归调用 dfs 函数。在主函数中，通过另一个双层循环检查 lawn 数组的每个位置。如果发现一个未访问的花区域（lawn[i][j] == 1 && visited[i][j] == 0），则增加花的种类计数 count，并调用 dfs 函数来标记这一种类的所有花。

【5、AC 截图及运行测试截图】



【1、tfs-最大团】

给定一个无向图 G=(V,E)。若 ,且对任意的,都有边 ,则称 U 是图 G 的一个完全子图。

G 的完全子图 U 是一个图，当且仅当 U 不包含在 G 的更大的完全子图中

G 的最大团是指包含顶点数最多的团。对给定的无向图，找出最大团中定点的个数。

【2、程序代码及注释】

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int G[50][50], bestn;
5. bool x[50]; //代表是否在云图中
6. int n;
7.
8. void Backtrack(int t, int n, int pn) {  //t 是当前正在考虑的的顶点的索引。n 是图的顶点总数,pn 是当前团中顶点的数量。
```

```
9.      if (t > n) {
10.         bestn = max(bestn, pn);
11.         return;
12.     }
13.
14.     bool op = true;    //op 用于进行剪枝操作的判断
15.     for (int i = 1; i < t; i++) {  //遍历先前的所有节点
16.         if (x[i] && G[t][i] == 0) {  //如果先前的顶点有加入至云图并且与当前节点不存在边的联系，则不符合条件，剪枝
17.             op = false;
18.             break;
19.         }
20.     }
21.
22.     if (op) {  //如果符合条件，当前考虑节点+1 ，云图节点+1
23.         x[t] = true;
24.         Backtrack(t + 1, n, pn + 1);
25.     }
26.
27.     if (pn + n - t > bestn) { //如果云图节点 + 剩余节点的和大于最大云图节点数，则还可以继续搜索云图，否则没有必要
28.         x[t] = false;
29.         Backtrack(t + 1, n, pn);
30.     }
31.}
32.
33.void get_graph(){
34. cin >> n;
35.     memset(G, 0, sizeof(G));
36.     for (int i = 1; i <= n; i++) {  //构建图
37.         for (int j = 1; j <= n; j++) {
38.             cin >> G[i][j];
39.         }
40.     }
41.}
42.
43.void solve(){
44. get_graph();
45. bestn = 0;       //初始化最大团节点
46.     memset(x, false, sizeof(x));
47.     Backtrack(1, n, 0);
48.     cout << bestn << endl;
49.}
```

```
50.
51.int main() {
52.  solve();
53.     return 0;
54.}
```

【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

首先我们对问题进行分析，最大团问题是寻找一个无向图中顶点数最多的完全子图，即图中的所有顶点之间都有边相连。首先进行输入处理，构建图表。接着再调用回溯函数 Backtrack，Backtrack 函数是回溯算法的核心，它尝试构建最大团。Backtrack 函数中 t 是当前正在考虑的顶点的索引，n 是图的顶点总数，pn 是当前团中顶点的数量。op 变量用于剪枝操作，如果当前顶点 t 与之前已经选择的顶点 i 之间没有边相连（即 G[t][i] == 0），则当前选择不是最大团的一部分，可以剪枝。数组 x 用于记录当前顶点是否已经加入到团中。当所有顶点都被考虑过后（t > n），比较当前团的大小 pn 与已知的最大团 bestn，并更新 bestn。

我们的搜索策略为如果当前顶点可以加入团中（op 为 true），则将其标记为已选择，并继续递归搜索下一个顶点。如果当前顶点不能加入团中，则首先判断是否有继续搜索的必要，若有则尝试不将其加入团中，并判断继续搜索。

【5、AC 截图及运行测试截图】

